

pkg/vm + Image-Cache Bundled Design (6.K-debt closure + Group C image-cache item)

Status: Design approved (operator, 2026-05-05 evening) after Group B closure (0ac633e). Implementation pending — subagent-driven execution per Group A-prime / Group B pattern. Branch: Containers lava-pin/2026-05-07-pkg-vm. Lava parent commits on master once the pin lands.

Forensic anchor. Group A-prime mechanically closed §6.N-debt; Group B added matrix-runner reliability + observability tightenings. What both left open is the §6.K constitutional debt — the requirement that submodules/containers/pkg/vm/ exist for QEMU full-system emulation, alongside the existing pkg/emulator/ (Android), so cross-architecture and cross-OS testing isn't blocked by the absence of a generic VM-orchestration package.

The §6.K-debt clause names four close criteria. Three are already met by Group A-prime + Group B:

- (1) pkg/emulator/ exists ✓ (Group A-prime + Group B)
- (3) Lava-side emulator-orchestration glue extracted ✓ (scripts/run-emulator-tests.sh is now thin glue)
- (4) scripts/check-constitution.sh updated for §6.N awareness ✓ (Group A-prime)

Only criterion (2) — "at least the QEMU baseline" in pkg/vm/ (or sibling) — remains open. This spec closes it AND bundles the only Group C reinforcement candidate that has natural overlap (image-cache management is a generic emulator-orchestration concern that both pkg/emulator/ and pkg/vm/ need). The other Group C candidates (network simulation, hardware-screenshot capture) stay deferred to a true Group C cycle later.

1. Scope

1.1 In scope (3 Containers packages + 2 Lava consumers + 1 closure)

1. **submodules/containers/pkg/cache/** (new) — content-addressable store for image artifacts (qcow2 for VMs, Android system-images for emulators). Project-agnostic: no specific images committed inside Containers. Caller supplies a manifest JSON declaring (id, url, sha256, size, format) tuples; pkg/cache/ fetches on cache miss, verifies SHA-256, refuses on mismatch.

2. **submodules/containers/pkg/vm/** (new) — QEMU full-system VM orchestration. VM interface { Boot, WaitForReady, Upload, Run, Download, Teardown }. VMMatrixRunner analog of AndroidMatrixRunner — emits the SAME attestation row schema (gating, diag, failure_summaries, concurrent) so scripts/tag.sh's 3 Group B gates work unchanged. KVM-where-available, TCG fallback. --concurrent N + --dev semantics identical to pkg/emulator/ (either flag flips gating: false).
3. **submodules/containers/cmd/vm-matrix** (new) — thin CLI wrapper mirroring cmd/emulator-matrix. Flags: --targets, --image-manifest, --uploads, --script, --captures, --evidence-dir, --concurrent, --dev, --boot-timeout, --script-timeout.
4. **submodules/containers/pkg/emulator/ refactor** — route Android system-image fetch through pkg/cache/ instead of the package-local download logic. External API of pkg/emulator/ UNCHANGED — the refactor is internal. Anti-bluff: ≥ 1 falsifiability rehearsal proving that the cache backend swap doesn't break the attestation row schema or break existing Group B gates.
5. **Lava cross-arch signing consumer** — Lava-side glue (scripts/run-vm-signing-matrix.sh + tests/vm-signing/) that drives cmd/vm-matrix against (Alpine/Debian/Fedora) $\times$ (x86_64/aarch64/riscv64) sub-matrix to verify Lava's signing/keystore code produces byte-equivalent signed APKs across all 9 configs. Bluff vector: JCA provider divergence (different signing bytes from different JREs on different arches). Test scope **A1 (sign-and-compare bytes)** per Q4. Each row records signing_match: bool derived from byte equivalence to the x86_64 KVM reference run.
6. **Lava cross-OS distro consumer** — Lava-side glue (scripts/run-vm-distro-matrix.sh + tests/vm-distro/) that drives cmd/vm-matrix against (Alpine/Debian/Fedora) $\times$ x86_64 to verify the Ktor proxy fat JAR + the lava-api-go static binary boot AND respond to a real functional request on each distro. Bluff vector: distro-specific runtime failure (libc differences, JRE provider differences, network stack differences). Test scope **B2 (boot-and-functional)** per Q4. Each row records proxy_health: bool, proxy_search: bool, goapi_health: bool, goapi_metrics: bool.
7. **Lava manifest + closure** — tools/lava-containers/vm-images.json (the project-side manifest declaring 9 base qcow2 images), Containers pin bump, .lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json, .lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json.

1.2 Out of scope (deferred to future cycles)

- Group C reinforcement candidates **other than image-cache management**: network simulation per AVD, hardware-screenshot capture per row.
- Non-baseline OS targets in pkg/vm/: minimal Windows for gradlew.bat parity, FreeBSD, macOS / iOS — all roadmap items in §6.K's KDoc.
- Image-build orchestration (Packer / cloud-init recipe-driven image generation) — pkg/vm/imagebuilder/ is a future cycle. v0.1 ships with operator-supplied URLs that point at upstream cloud images.
- Pluggable Cache backend interface — YAGNI for v0.1 (filesystem CAS only). Can be retrofitted if a second backend ever appears.

- VM-only tag.sh gates — scripts/tag.sh handles VM matrix attestations transparently via the existing find <pack> -name 'real-device-verification.json' pattern, since pkg/vm/s writeAttestation emits the same schema.

1.3 Non-goals

- **Not** a re-design of pkg/emulator/. The refactor in this spec is mechanical (route image-fetch through pkg/cache/); the matrix-runner gate semantics + Group B reliability fixes stay as-is.
- **Not** a relaxation of any clause. v0.1 strengthens: cross-arch and cross-OS testing become possible from the project's own repo, where today they're not.
- **Not** a Lava-only change. Components 1-4 live in Containers because they're generic capabilities the next vasic-digital project to use VM orchestration would want identically. Components 5-7 live in Lava because the specific consumer matrices + the manifest content are Lava-domain (per Decoupled Reusable Architecture).

2. Architecture



	pkg/	
vm/		
	types.go	VM, VMTarget, BootResult, VMConfig,
VMMatrixConfig,		VMMatrixResult, with the SAME row-extension
fields		as pkg/emulator (gating, diag,
failure_summaries,		
concurrent)		
	qemu.go	QEMUVM impl of VM iface; KVM/TCG auto-
detect by		target arch + /dev/kvm presence; ssh client
over		golang.org/x/crypto/ssh; SCP via ssh; QMP
control-		socket for graceful
shutdown		
	matrix.go	VMMatrixRunner - runOne extracted;
captureDiag		
		between Boot and Run; worker pool
when		Concurrent>1; writeAttestation emits same
schema		
	teardown.go	Teardown{30s SSH/QMP grace then
KillByPort		fast-path reusing pkg/emulator/
cleanup.KillByPort		on the qemu monitor
port}		
	qemu_test.go	Tests with fake SSH/QMP/exec
executors		
	matrix_test.go	Tests with stubVM mirror of
stubEmulator		
	teardown_test.go	Tests using killByPortHook
seam		
	cmd/vm-	
matrix/		
	main.go	CLI wrapper; flag parser; dispatches
to		
		pkg/vm.NewQEMUMatrixRunner; same exit-
code		
		contract as cmd/emulator-matrix (0 green,
1		failed, 2 invalid
args)		
	pkg/	
emulator/		



```

|
tests/
|
|   └─ vm-
signing/
|
|   test_signing_matrix_rejects_byte_divergence.sh
|
|   test_signing_matrix_accepts_concordant_signatures.sh
|   └─ vm-
distro/
|
test_distro_matrix_rejects_proxy_health_failure.sh
test_distro_matrix_rejects_goapi_metrics_failure.sh
test_distro_matrix_accepts_clean_run.sh
|
|   .lava-ci-
evidence/
|   └─ bluff-hunt/2026-05-07-pkg-
vm.json
|   └─ Phase-pkg-vm-
closure-2026-05-07.json
|
|
CLAUDE.md
|
|   (~) $6.K-debt RESOLVED paragraph appended (mirrors $6.N-
debt
|   RESOLVED pattern from Group A-prime; non-load-
bearing)
|

```

3. Components

3.1 pkg/cache/ (Containers)

Manifest schema (pkg/cache/manifest.go):

```

// Manifest declares a project's pinned image artifacts. Lives
// outside
// Containers (Lava ships its at tools/lava-containers/vm-
// images.json).
// pkg/cache/ defines the schema; consumers populate it.
type Manifest struct {
    Version int           `json:"version" // currently 1
    Images  []ImageEntry         `json:"images"`
}

```

```

type ImageEntry struct {
    ID      string `json:"id"`      // canonical id, e.g.
             "alpine-3.20-x86_64"
    URL      string `json:"url"`      // source URL the cache fetches
             on miss
    SHA256    string `json:"sha256"`  // hex-encoded, REQUIRED
    Size     int64  `json:"size"`    // bytes, REQUIRED – sanity-
             check the download
    Format    string `json:"format"`  // "qcow2" | "android-system-
             image" | "raw"
}

// LoadManifest parses + validates a manifest file. Returns error
// on
// duplicate IDs, malformed SHA256, or schema-version mismatch.
func LoadManifest(path string) (*Manifest, error)

Store contract (pkg/cache/store.go):

// Store is the content-addressable cache.
// Cache root: $XDG_CACHE_HOME/vasic-digital/containers-images/
// (default
// ~/.cache/vasic-digital/containers-images/). Layout:
//
// blobs/sha256/<full-hash> → image bytes (read-only)
// lockfiles/<sha-prefix>.lock → flock per-image for concurrent
// download
//
// Get returns the local path of the image's bytes. On cache miss,
// fetch
// from URL + verify SHA-256; on mismatch, the partial download is
// removed and a non-nil error is returned. Concurrent Get() calls
// for
// the same image serialize on the per-image lockfile.
type Store interface {
    Get(ctx context.Context, m *Manifest, imageID string) (path
        string, err error)
    Verify(ctx context.Context, m *Manifest, imageID string) error
    Refresh(ctx context.Context, m *Manifest, imageID string)
        error
}

```

Anti-bluff posture (Sixth Law clause 2): every public function in pkg/cache/ MUST have a falsifiability-rehearsal test. Canonical rehearsals:

- **TestStore_SHA256Mismatch_RejectsAndRemovesBlob** — replace the blob with one whose SHA differs; assert Get returns an error AND the bad blob is removed.
- **TestStore_ConcurrentGet_SerializesViaFlock** — two goroutines call Get on the same imageID; both succeed; the URL is fetched only once (use a counting fake HTTP server).

3.2 pkg/vm/ (Containers)

VM interface (pkg/vm/types.go):

```
type VMTarget struct {
    ID      string `json:"id"`           // matches an ImageEntry.ID
    Arch    string `json:"arch"`         // "x86_64" | "aarch64" |
                        "riscv64"
    Distro  string `json:"distro"`       // "alpine" | "debian" |
                        "fedora"
    Version string `json:"version"`
}

type BootResult struct {
    Target      VMTarget
    Started     bool
    SSHPort     int           // host port forwarded to guest:22
    MonitorPort int           // host port for QMP control
                socket
    BootDuration time.Duration
    Error         error
}

type VM interface {
    Boot(ctx context.Context, target VMTarget, qcow2Path string)
        (BootResult, error)
    WaitForReady(ctx context.Context, sshPort int, timeout
        time.Duration) error
    Upload(ctx context.Context, sshPort int, hostPath, vmPath
        string) error
    Run(ctx context.Context, sshPort int, command string, env
        map[string]string, timeout time.Duration) (stdout, stderr
        string, exitCode int, err error)
    Download(ctx context.Context, sshPort int, vmPath, hostPath
        string) error
    Teardown(ctx context.Context, monitorPort, sshPort int) error
}
```

VMMatrixRunner (pkg/vm/matrix.go): identical shape to pkg/emulator/
AndroidMatrixRunner — runOne extracted; per-target row populated with Diag
(target/distro/uname-r/ssh-version), FailureSummaries (parsed from script's stderr/
exit-code), Concurrent (the matrix runner's setting). MatrixResult.Gating =
concurrent == 1 && !config.Dev.

Teardown fast-path (pkg/vm/teardown.go): SSH poweroff + 30s graceful grace,
then port-strict KillByPort(monitorPort) reusing pkg/emulator/
cleanup.go::KillByPort (already strict-adjacent token matcher; no re-
implementation). Skip-on-mismatch identical to pkg/emulator's.

3.3 cmd/vm-matrix/ (Containers)

Thin CLI wrapper. Flags:

<code>--image-manifest <path></code>	Path to <code>vm-images.json</code>
<code>--targets <list></code>	Comma-separated list of ImageIDs from manifest
<code>--uploads <list></code>	Comma-separated host:vm path pairs to upload
<code>--script <path></code>	Host-path to shell script run on each target
<code>--captures <list></code>	Comma-separated vm:host path pairs to download
<code>--evidence-dir <path></code>	Per-target attestation row directory
<code>--concurrent <N></code>	Default 1
<code>--dev</code>	Permits snapshot reload; sets <code>Gating=false</code>
<code>--boot-timeout <duration></code>	Default per-arch (60s/240s/360s – see D6)
<code>--script-timeout <duration></code>	Default 10m

Exit codes match `cmd/emulator-matrix`: 0 = matrix passed, 1 = at least one target failed, 2 = invalid args / runner error.

3.4 pkg/emulator/ refactor (Containers)

Goal: route Android system-image fetch through `pkg/cache/`. **External API unchanged.**

Today, `pkg/emulator/android.go:Boot` consumes the operator's `ANDROID_SDK_ROOT/system-images/`. The refactor: when an AVD's required system-image is absent, `pkg/emulator/` invokes `pkg/cache/Store.Get(manifest, "android-<api>-<abi>")` instead of failing or instructing the operator to run `sdksmanager`. The Lava-side manifest at `tools/lava-containers/vm-images.json` gains `android-*` entries alongside the `qcow2` entries.

Anti-bluff posture: ≥ 1 falsifiability rehearsal —

`TestAndroidEmulator_Boot_FetchesMissingSystemImageViaCache_AndDoesNotChangeAttestationSchema`. Mutation: route to a synthetic cache that returns a different image path → assert (a) Boot still succeeds AND (b) attestation row schema is byte-equivalent to the pre-refactor schema.

3.5 Cross-arch signing consumer (Lava parent)

`scripts/run-vm-signing-matrix.sh` invokes `cmd/vm-matrix` with:

- `--targets`: 9 IDs, all combinations of (alpine,debian,fedora) × (x86_64,aarch64,riscv64)
- `--uploads`: `proxy/build/libs/app.jar`, `keystores/upload.keystore.p12`, `tests/vm-signing/sample.apk`
- `--script`: `tests/vm-signing/sign-and-hash.sh` (runs `jarsigner + sha256sum signed.apk`)
- `--captures`: `/tmp/signed.apk`, `/tmp/signing-output.json`

Post-processing: read each target's `signing-output.json` for the SHA-256 of the signed bytes. Compare each to the x86_64 KVM reference's hash. Row gets `signing_match: true` iff hashes match (byte-equivalent signing).

Acceptance criterion: row's `signing_match` is true for all 9 targets. Any divergence is the JCA-provider bluff Sixth Law clause 1 was written to catch.

Falsifiability rehearsal (Lava Phase C):

`test_signing_matrix_rejects_byte_divergence.sh` — fixture seeds a synthetic attestation with one row's hash differing → assert post-processing rejects.

3.6 Cross-OS distro consumer (Lava parent)

`scripts/run-vm-distro-matrix.sh` invokes `cmd/vm-matrix` with:

- `--targets`: 3 IDs (`alpine-x86_64`, `debian-x86_64`, `fedora-x86_64`) — distro coverage not arch
- `--uploads`: `proxy/build/libs/app.jar`, `lava-api-go/build/lava-api-go`
- `--script`: `tests/vm-distro/boot-and-probe.sh` (starts both, polls health + functional endpoints)
- `--captures`: `/tmp/probe-output.json`

Post-processing: read each target's `probe-output.json` for `proxy_health: bool`, `proxy_search: bool`, `goapi_health: bool`, `goapi_metrics: bool`. All four must be true.

Acceptance criterion: all 4 booleans are true for all 3 distros.

Falsifiability rehearsals (Lava Phase C):

- `test_distro_matrix_rejects_proxy_health_failure.sh` — fixture with one distro reporting `proxy_health: false` → assert rejected.
- `test_distro_matrix_rejects_goapi_metrics_failure.sh` — analogous.
- `test_distro_matrix_accepts_clean_run.sh` — golden path.

3.7 Lava manifest + closure

tools/lava-containers/vm-images.json: 9 qcow2 entries (`alpine/debian/fedora × x86_64/aarch64/riscv64`) + at least 1 Android system-image entry referenced by the `pkg/emulator/` refactor. SHA-256 + size + URL committed; bumps require deliberate operator review.

Closure attestation at `.lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json` — same shape as Group B closure (per-component SHAs, mutation-rehearsal record, mirror convergence verified via `live git ls-remote`, "next group preview" line).

Bluff-hunt at `.lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json` — §6.N.1.1 lighter same-day hunt covering 1-2 production-code files (`pkg/cache/store.go::Get` + `pkg/vm/qemu.go::Boot`).

4. Data flow

4.1 Cross-arch signing run (the canonical Lava consumer path)

```
operator: ./scripts/run-vm-signing-matrix.sh
  └─ parse tools/lava-containers/vm-images.json
  └─ invoke cmd/vm-matrix with 9 targets, uploads, script,
captures
  └─
cmd/vm-matrix:
  └─ for each target (sequentially in --concurrent=1):
    └─ pkg/cache/Store.Get(manifest, target.ID) → /home/.../
blobs/sha256/<hash>
    └─ pkg/vm/QEMUVM.Boot(target, qcow2_path) → SSH/QMP ports
    └─ pkg/vm/QEMUVM.WaitForReady (SSH listener up)
    └─ captureDiag → DiagnosticInfo{ id, arch, distro, uname-r,
ssh-banner }
    └─ for each upload: pkg/vm/QEMUVM.Upload
    └─ pkg/vm/QEMUVM.Run(script) → (stdout, stderr, exitCode)
    └─ for each capture: pkg/vm/QEMUVM.Download
    └─ parse stderr / exitCode for FailureSummaries
    └─ pkg/vm/QEMUVM.TearDown
    └─ append AVDRow-equivalent to MatrixResult.Rows
  └─ writeAttestation → real-device-verification.json with
gating: true
  └─ exit 0 if all rows passed
  └─
post-processing (run-vm-signing-matrix.sh, Lava-side):
  └─ for each row: read captured signing-output.json → SHA-256
  └─ compare each to the x86_64 KVM reference
  └─ augment attestation with signing_match bool per row

operator: scripts/tag.sh Lava-Android-X.Y.Z-NN
  └─ existing clause-6.I-clause-7 helper finds real-device-
verification.json
  └─ existing 3 Group B gates (concurrent=1, gating=true,
diag.sdk-or-id consistency)
  └─ tag created (assuming all green)
```

4.2 Image cache miss (fetch + verify)

```
pkg/cache/Store.Get(ctx, manifest, "alpine-3.20-x86_64"):
  └─ lookup ImageEntry by ID
  └─ compute target path: $CACHE/blobs/sha256/<entry.sha256>
  └─ if path exists: stat-then-return (fast path)
  └─ acquire flock on $CACHE/lockfiles/<sha-prefix>.lock
    (concurrent Get for same ID → second caller waits, then sees
the
  └─ blob in the fast path)
  └─ HTTP GET entry.URL → tempfile
  └─ stream-hash with sha256.New() while writing
  └─ if computed_sha != entry.SHA256: rm tempfile, return error
```

```

    |— if entry.Size != 0 AND tempfile.size != entry.Size: rm,
return error
    |— atomic rename tempfile → final path
    |— release flock
    |— return final path

```

4.3 VM matrix Teardown fast-path

```

pkg/vm/QEMUVM.TearDown(ctx, monitorPort, sshPort):
    |— SSH "poweroff" via existing sshPort connection
    |— poll QMP socket for "SHUTDOWN" event up to
teardownGracePeriod (30s)
    |   (graceful Linux shutdown completes here; QEMU exits)
    |— if QEMU still alive after grace:
    |   |— pkg/emulator/cleanup.KillByPort(monitorPort) – strict-
adjacent /proc walk
    |   |— if Matched=0: return "did not exit; KillByPort matched 0
(skip-on-mismatch safety)"
    |   |— if Matched>0: re-poll for /proc clearing (5s); return
nil if gone
    |— return nil on graceful exit

```

The fast-path reuses `pkg/emulator/cleanup.go::KillByPort` directly — same strict-adjacent matcher, same skip-on-mismatch safety, no re-implementation. Lava Phase C's falsifiability rehearsal explicitly proves this code path:

`TestVMTearDown_FastPath_SkipsOnMismatch` is a structural copy of `TestTearDown_FastPath_SkipsOnMismatch` from Group B Phase A.

5. Error handling

Scenario	Behavior	Why
<code>pkg/cache.Store.Get</code> SHA mismatch	Remove partial blob, return error	Bluff vector — silent corrupt cache hit is exactly what §6.J forbids
<code>pkg/cache.Store.Get</code> HTTP error	Return error, don't poison cache	Network blips are recoverable on retry
<code>pkg/cache</code> concurrent Get for same image	Serialize on per-image flock; both succeed; URL fetched once	Avoid 2× download on parallel matrix runs
<code>pkg/vm.QEMUVM.Boot</code> KVM unavailable on x86_64	Fall through to TCG; log "[vm] KVM unavailable, using TCG (slow)"	Operator transparency; doesn't fail
<code>pkg/vm.QEMUVM.Boot</code> qemu-system- not on PATH	Return error per-target; matrix continues to next target	Per-target row failure ≠ matrix-runner crash
	Return error; matrix-row Failed	

Scenario	Behavior	Why
pkg/ vm.QEMUVM.WaitForReady SSH not listening past timeout		Honest signal — boot succeeded but guest never came up
pkg/vm.QEMUVM.Run script exits non-zero	Capture stdout/stderr/ exitCode; row Passed=false	Standard test-failure path
pkg/vm.QEMUVM.Upload SSH copy fails	Return error; matrix- row Failed	Same as Run
pkg/vm.QEMUVM.Teardown graceful grace expires	KillByPort fast-path on monitor port	Group B Teardown pattern
cmd/vm-matrix -- concurrent>1 + script writes to shared host dir	Document as "scripts must be concurrency- aware"; matrix runner does not enforce	Same as pkg/ emulator's TestReportGlob warning
Lava signing consumer: signing_match false on >0 row	Reject the run; tag.sh refuses (assuming row.Passed=false too)	JCA bluff caught
Lava distro consumer: any of 4 booleans false	Reject the run; tag.sh refuses	Distro bluff caught

6. Testing

Layer	What	Real-stack?	Falsifiability rehearsal?
pkg/cache/*_test.go (Containers)	Manifest parse, SHA verify, flock concurrency, refresh-after- mutation	Real os + real flock + counting-fake HTTP server	YES — drop SHA verify → mismatch test fails
pkg/vm/qemu_test.go (Containers)	Boot/ WaitForReady/ Upload/Run/ Download/ Teardown via fake exec/SSH executors	Injected fakes at the boundary	YES — drop SSH-port- strict match → race- condition test fails
pkg/vm/matrix_test.go (Containers)	runOne, captureDiag, FailureSummaries parsing, worker pool, gating-flag flip	stubVM (mirror of stubEmulator)	YES — flip Gating default → defaults test fails
pkg/vm/teardown_test.go (Containers)	KillByPort fast- path under skip- on-mismatch safety; reuses pkg/	Injected killByPortHook seam	YES — drop skip-on- mismatch return →

Layer	What	Real-stack?	Falsifiability rehearsal?
	emulator/ cleanup.KillByPort		safety test fails
pkg/emulator/*_test.go (Phase B refactor)	Existing 41 tests + 1 new (cache- routed Boot doesn't change attestation schema)	Same fakes as Group B	YES — replace cache backend with one returning a different path → schema- equality test fails
tests/vm-signing/ test_signing_matrix_*.sh (Lava)	tag.sh-equivalent rejection of byte- divergent signing rows	Real bash + jq + fixture attestation JSON	YES — drop signing_match check → divergence test fails
tests/vm-distro/ test_distro_matrix_*.sh (Lava)	tag.sh-equivalent rejection of proxy_health/ goapi_metrics false rows	Same	YES — drop boolean check → failure- injection test fails
Real-stack matrix run (operator)	./scripts/run- vm-signing- matrix.sh (real qemu-system-* + real SSH + real qcow2 boot) — produces a real attestation with signing_match: true × 9; analogous for distro consumer	YES — real cold-boot QEMU, real artifact upload, real script execution	NOT applicable to a single test — but the matrix run itself satisfies clause 6.K + 6.I

Falsifiability rehearsal floor per phase (per Q8 D5):

- **Phase A (Containers, all 3 packages + cmd):** ≥5 rehearsals.
- **Phase B (Containers, emulator refactor):** ≥1 rehearsal.
- **Phase C (Lava consumers):** ≥2 rehearsals (1 per consumer).
- **Phase D (closure):** none (evidence-only, like Group B Phase C).

7. Order of operations

Phase	Branch	Files (high-level)	Commits
A. Containers code	lava-pin/ 2026-05-07- pkg-vm	pkg/cache/* (new), pkg/vm/* (new), cmd/vm-matrix/main.go (new)	1 commit; subagent dispatched per Group

Phase	Branch	Files (high-level)	Commits
			A-prime / B pattern
B. Emulator refactor	same Containers branch	pkg/emulator/android.go (cache-routed image fetch), pkg/emulator/types.go (if MatrixConfig gains an ImageManifest path field)	1 commit
C. Lava consumers	Lava master	tools/lava-containers/vm-images.json, scripts/run-vm-{signing,distro}-matrix.sh, tests/vm-{signing,distro}/*.sh, CLAUDE.md (§6.K-debt RESOLVED)	1 commit
D. Pin bump + closure	Lava master	submodules/containers SHA bump, .lava-ci-evidence/bluff-hunt/2026-05-07-pkg-vm.json, .lava-ci-evidence/Phase-pkg-vm-closure-2026-05-07.json	1 commit

Total: **2 Containers commits + 2 Lava parent commits**. Mirror push to all 4 Lava upstreams + 2 Containers upstreams (github + gitlab — gitflic + gitverse aren't configured for the Containers submodule, matching inherited baseline) after each commit; convergence verified via live `git ls-remote`.

Estimated wall-clock: 8-12 hours (Phase A is the bulk — pkg/cache + pkg/vm + cmd/vm-matrix is roughly 2-3× the surface area of Group B Phase A).

8. Constitutional bindings

- **Sixth Law clauses 1-5.** Every test traverses real production code paths; falsifiability rehearsal recorded per test class; primary assertion on user-visible state (cache file path, VM exit code, attestation row schema, `signing_match` bool, `proxy_health` bool).
- **Sixth Law clause 6 (inheritance).** All new files inherit the Pact recursively into Containers and Lava both.
- **Seventh Law clause 1 (Bluff-Audit stamp).** Every commit's body carries the test/mutation/observed/reverted block per §6.N.1.2 (pre-push Check 4 enforces).
- **Clause 6.I (Multi-Emulator Container Matrix).** pkg/vm's `VMMatrixRunner` emits the same attestation row schema as pkg/emulator's `AndroidMatrixRunner` — `tag.sh`'s 3 Group B gates apply transparently.
- **Clause 6.J / 6.L (Anti-Bluff Functional Reality Mandate).** Every gate exists for one reason: refuse evidence that doesn't represent a real-user-completes-the-flow run. The cross-arch signing consumer specifically catches the JCA bluff; the cross-OS distro consumer catches distro-specific runtime divergence.
- **Clause 6.K (Builds-Inside-Containers Mandate).** This spec **closes 6.K-debt**: criterion (2), the QEMU baseline in pkg/vm/, lands. Criteria (1) (3) (4) were already met by Group A-prime + Group B.
- **Clause 6.M (Host-Stability Forensic Discipline).** pkg/vm. `QEMUVM.TearDown`'s `KillByPort` fast-path reuses pkg/emulator/

`cleanup.KillByPort` — strict-adjacent matcher, no broad pkill, `Matched=0` is no-op-safe. Per-VM port-strict targeting means concurrent matrix runs / sibling-project QEMUs are NEVER touched.

- **Clause 6.N (Bluff-Hunt Cadence).** Phase D's bluff-hunt JSON is the §6.N.1.1 same-day record. Pre-push Check 5 mechanically validates it.
- **Decoupled Reusable Architecture.** `pkg/cache`, `pkg/vm`, `cmd/vm-matrix` are project-agnostic. The Lava-domain manifest (`tools/lava-containers/vm-images.json`) lives in Lava. Same separation pattern as `--test-report-glob` (Containers field) vs `app/build/outputs/...` (Lava-side passthrough).
- **Local-Only CI/CD.** No new hosted-CI surface. All gates are pre-push hooks + `tag.sh` + the operator's `./scripts/run-vm-*matrix.sh` invocation.

9. Mirror policy

Per §6.C (mirror-state mismatch checks) and Decoupled Reusable Architecture:

- Containers branch `lava-pin/2026-05-07-pkg-vm` MUST converge at the same SHA on its 2 configured upstreams (github + gitlab) before the Lava parent's pin-bump commit references it.
- Lava parent commits (Phases C and D) MUST converge at the same SHA on all 4 upstreams before the closure attestation is finalized.
- Convergence verified via live `git ls-remote` — NOT via stale `.git/refs/remotes/<r>/master` cache. Group A-prime + Group B reviewer iterations recorded false-positive divergences from misreading that cache.

10. Self-review checklist



Placeholder scan. No "TBD", no "TODO", no "implement later". Every component (3.1-3.7) has interface, file paths, anti-bluff posture detail.



Internal consistency. Architecture diagram, components section, data flow, testing table, order-of-operations all reference the same 4 phases, the same 3 Containers packages + 1 refactor + 2 Lava consumers, the same branch name (`lava-pin/2026-05-07-pkg-vm`), the same gating semantics (`Concurrent==1 && !Dev`).



Scope check. Single bundled increment closing 6.K-debt + the 1 Group C item with overlap. The other 2 Group C candidates (network sim, screenshot capture) explicitly out of scope. Phase decomposition into 4 commits keeps each shippable independently.



Ambiguity check. Every "MUST" / "MAY" usage intentional. The skip-on-mismatch safety is stated three times in three contexts (component 3.2 Teardown, error handling table, constitutional binding 6.M) precisely because Group B operator feedback ("safest approach, other processes may be used by other projects") needs zero room for misreading. The Decoupled Reusable Architecture separation (manifest in Lava, mechanism in Containers) is stated four times (sections 1.3, 3.1, 3.7, 8) for the same reason.

Spec status: approved (operator, 2026-05-05 evening). Awaiting operator review of this written form before transitioning to superpowers:writing-plans for the implementation plan.